



电子科技大学

University of Electronic Science and Technology of China

信息与软件工程学院

SCHOOL OF INFORMATION AND SOFTWARE ENGINEERING

W1D3-C语言知识强化：指针与链表

Linux下Web Server的设计

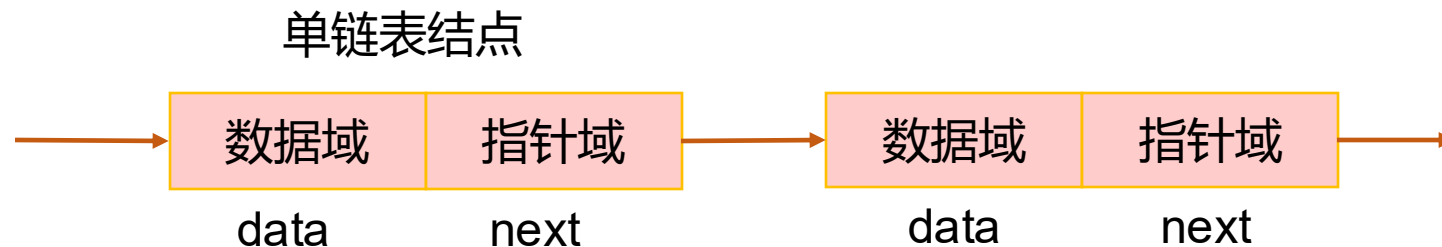
动态存储分配

序号	函数和描述
1	void *malloc(int num); 在堆区分配一块指定大小的内存空间，用来存放数据。这块内存空间在函数执行完成后不会被初始化，它们的值是未知的。
2	void free(void *address); 该函数释放 address 所指向的内存块,释放的是动态分配的内存空间。
3	void *calloc(int num, int size); 在内存中动态地分配 num 个长度为 size 的连续空间，并将每一个字节都初始化为 0。所以它的结果是分配了 num*size 个字节长度的内存空间，并且每个字节的值都是 0。
4	void *realloc(void *address, int newsize); 该函数重新分配内存，把内存扩展到 newsize。

注意：void * 类型表示未确定类型的指针。



单链表的存储结构描述



```
typedef struct Node {      /* 结点类型定义 */
```

```
    ElemType data;
```

```
    struct Node *next;
```

嵌套定义，指向自身类型的指针

```
}Node, *LinkList; /* LinkList为结构指针类型*/
```



单链表的存储结构描述

- 单链表的存储结构

```
typedef struct Node {    /* 结点类型定义 */  
    ElemType data; /* ElemType需要替换成实际数据类型 */  
    struct Node *next;  
}Node, *LinkedList; /* LinkedList为结构指针类型*/
```

- 定义链表L

LinkedList L; //一般不用Node*L

- ☺ 定义结点指针p;

Node * p; //一般不使用LinkedList p;

- **LinkedList 和 Node *** 同为结构指针类型
- **LinkedList** 一般用作单链表的头指针变量
- **Node *** 一般用作指向单链表中结点的指针



单链表的存储结构举例

001	张三	女
002	李四	男
003	王五	男
004	小明	女
.....		

```
typedef struct{           //学生信息的定义
    int id;                //学号
    char name[20];         //名字
    int sex;               //性别
}Student;
```

```
typedef struct Node {
    Student student ;
    struct Node *next;
}Node, *LinkList;
```



单链表的操作

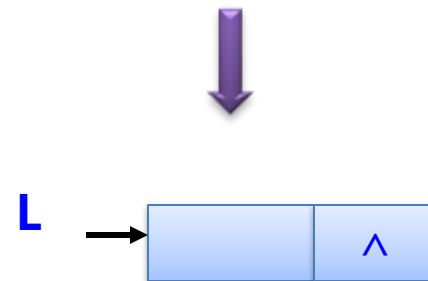
- 单链表的基本操作：
 - 初始化单链表
 - 判断单链表是否为空
 - 单链表的销毁
 - 求单链表的长度
 - 单链表的建立
 - 头插法建表
 - 尾插法建表
 - 单链表查找
 - 单链表的插入
 - 单链表的删除



单链表的基本运算--初始化单链表

初始化单链表算法步骤：

- 1、生成新结点作为头结点，并让L指向它。
- 2、将头结点的指针域置为空。



```
void initList(LinkList *L) {    /* L 实际上是双指针，指向指针的指针 */  
    *L=(LinkList)malloc(sizeof(Node)); /* 申请空间，建立头结点 */  
    (*L)->next=NULL;             /* 建立空的单链表 */  
}
```



单链表的基本运算--初始化单链表

```
typedef struct Node {    /* 定义结点类型 */
    ElemType data;
    struct Node *next;
}Node, *LinkList;      /* LinkList为结构指针类型*/

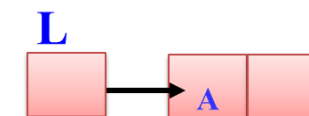
int main(){
    LinkList L = NULL;    /*定义头指针*/
    initList(&L);          /*初始化链表，定义头结点，并让L指向头结点*/
}

void initList(LinkList *L) {    /* 初始化链表，L为指向指针的指针*/
    *L=(LinkList)malloc(sizeof(Node)); /* 申请空间，建立头结点 */
    (*L)->next=NULL;          /* 建立空的单链表 */
}
```



单链表的基本运算—判断单链表是否为空

- 不带头节点的单链表：L指向单链表的第一个结点
 - 若 $L == \text{NULL}$ ，表示单链表为一个空表
 - 非空表，则 L 指向第一个元素结点（首元结点）
- 带头节点的单链表：L指向单链表的头结点
 - 若 $L \rightarrow \text{next} == \text{NULL}$ ，表示单链表为一个空表
 - 非空表，则 $L \rightarrow \text{next}$ 指向首元结点



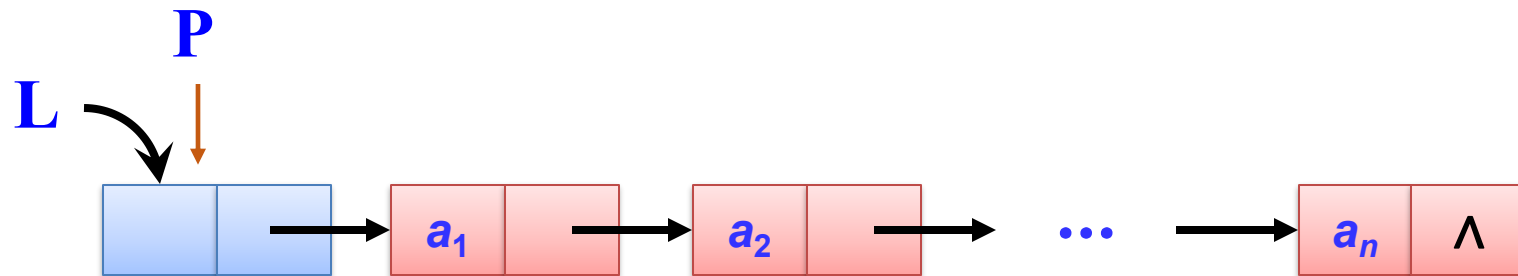
```
int isEmptyList(LinkList L) {  
    if (L->next)  
        return 0;  
    return 1;  
}
```

后面示例使用带头节点的单链表

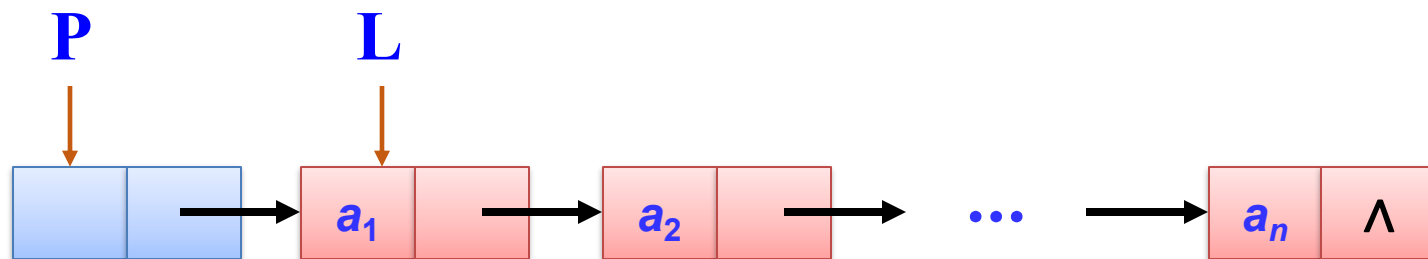


单链表的基本运算—单链表的销毁

- 算法思路：从头指针开始，依次释放所有结点。



$p=L;$

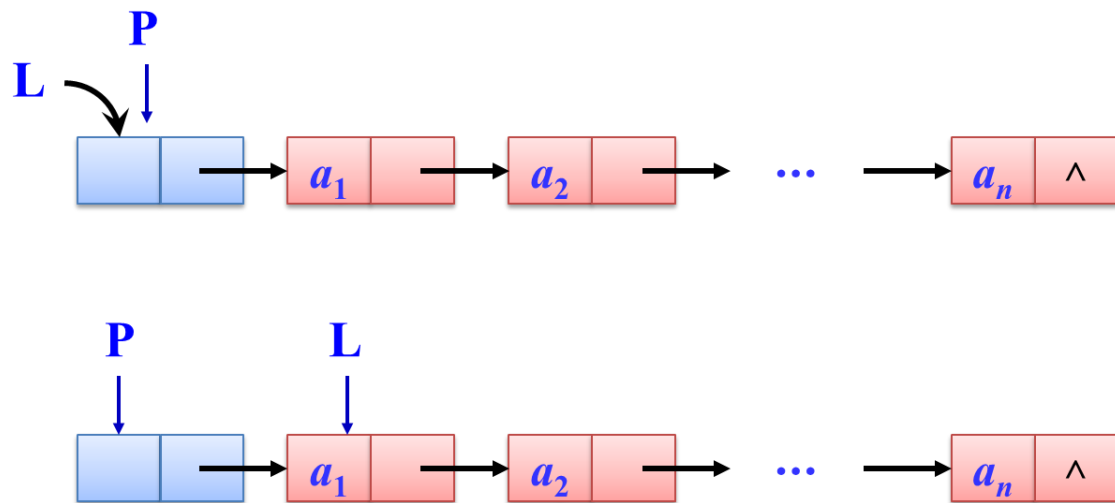


$L = L \rightarrow \text{next};$
 $\text{free}(p);$

循环条件: $L \neq \text{NULL}$



单链表的基本运算—单链表的销毁



销毁单链表的步骤：

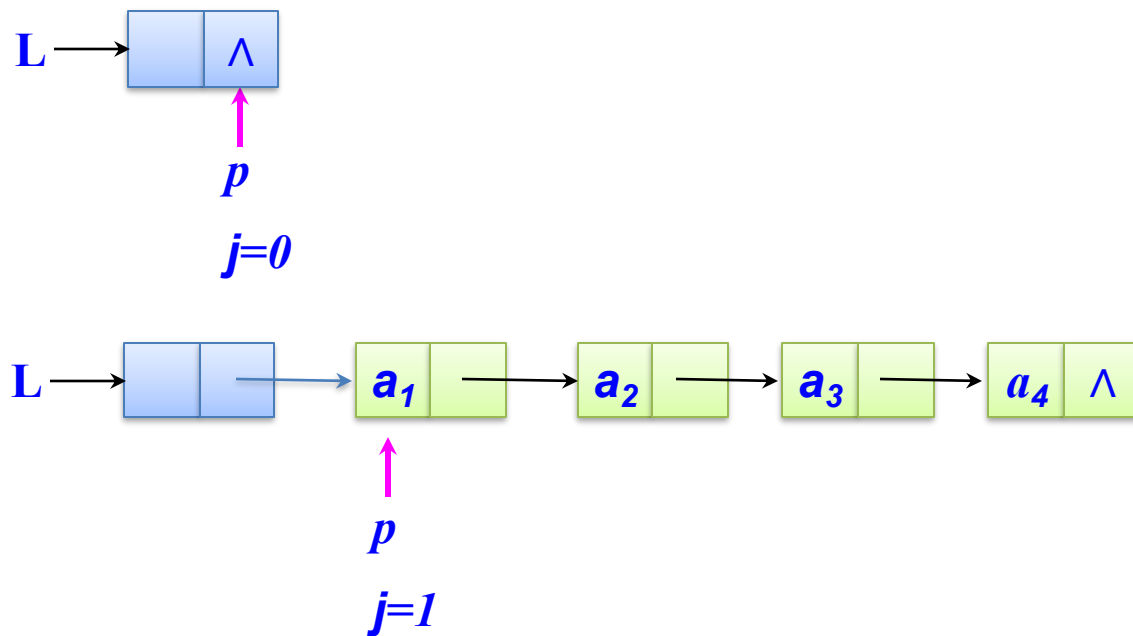
- 1、让指针 p 执行头指针 L 。
- 2、 L 指向 L 的下一个结点。
- 3、销毁 p 指向的结点。
- 4、反复执行1-3步，直至 L 为空。

```
int destoryList(LinkList L) {  
    Node * p;  
    while(L) {  
        p = L;  
        L = L->next;  
        free(p);  
    }  
    return 0;  
}
```



单链表的基本运算—求单链表的长度

```
int listLength(LinkList L) { /* 求带头结点的单链表L的长度 */  
    Node *p = L->next; /* p指向头结点的下一个结点 */  
    int j=0;           /* 用来存放单链表的长度 */  
    while(p!=NULL) {  
        j++;  
        p=p->next;  
    }  
    return j; /* j为求得的单链表长度 */  
}
```

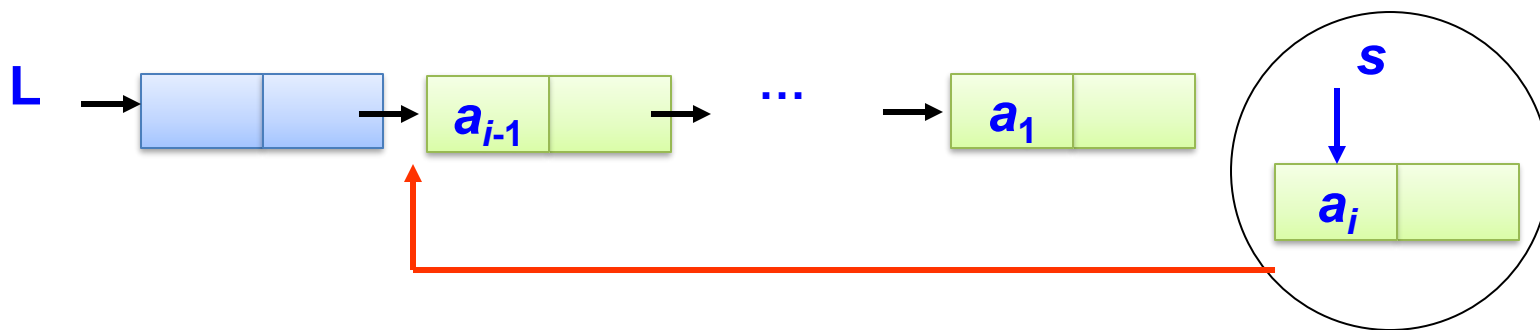


算法的时间复杂度为 **$O(n)$**



单链表的基本运算—头插法建表

- 从一个空表开始，创建一个头结点。
- 依次读取数据，生成新结点
- 将新结点插入到当前链表的表头上，直到结束为止。

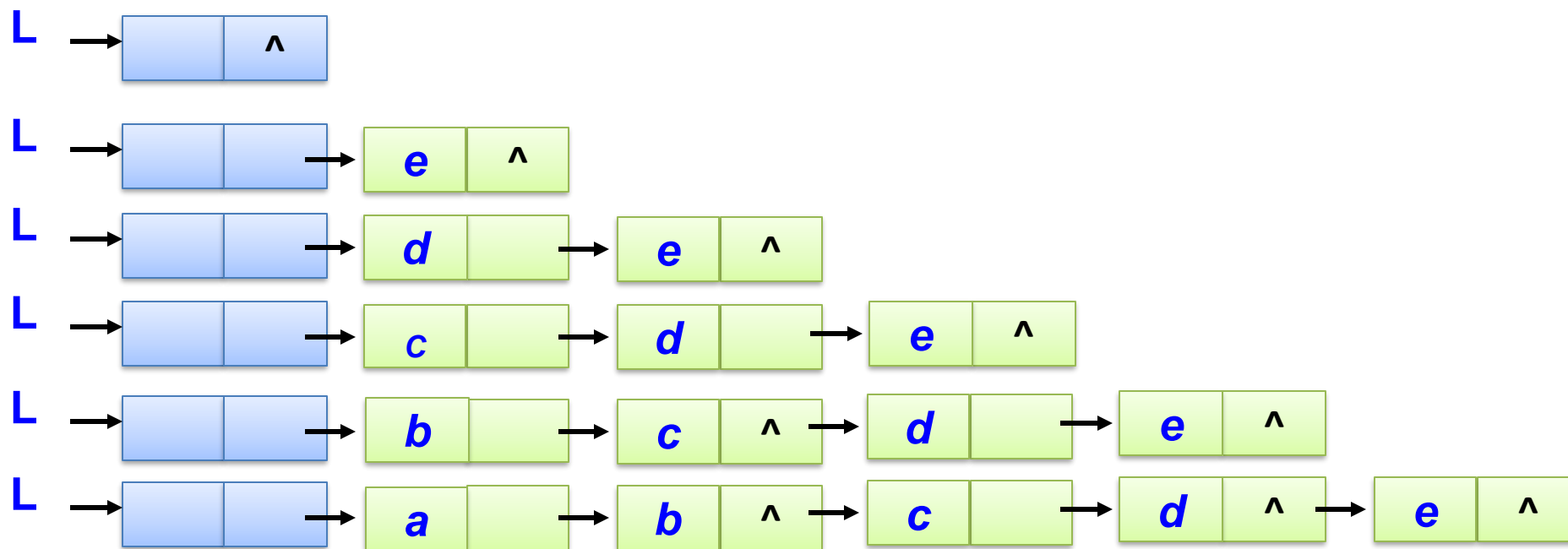


注意：链表的结点顺序与逻辑次序相反。



单链表的基本运算—头插法建表

如果需要建立 (a,b,c,d,e) 的链表
则需要输入: e d c b a \$



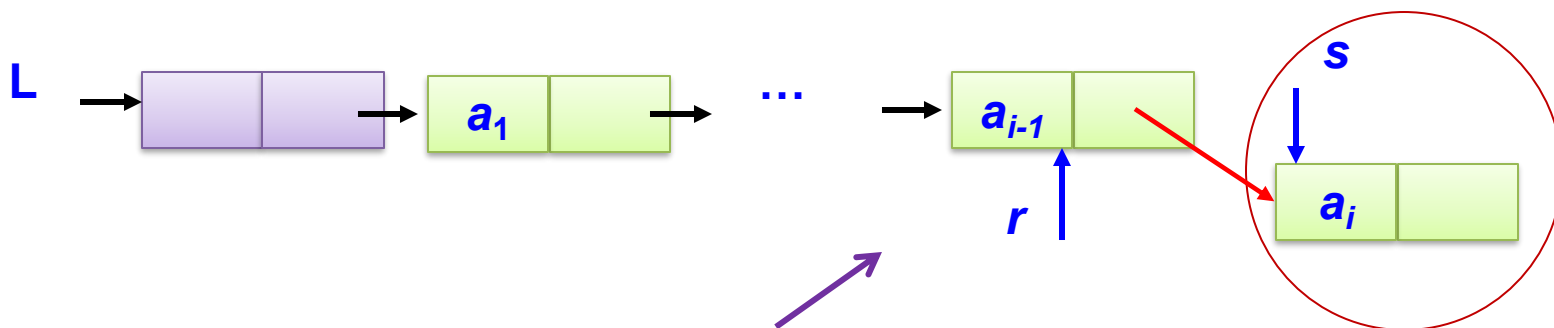
单链表的基本运算—头插法建表

```
void CreateFromHead(LinkList L) {  
    Node *s;  
    char c;  
    int flag=1;  
    while(flag) { /* flag初值为1, 当输入"$"时, 置flag为0, 建表结束*/  
        c=getchar();  
        if(c!='$') {  
            s=(Node*)malloc(sizeof(Node)); /*建立新结点s*/  
            s->data=c;  
            s->next=L->next; /*将s结点插入表头*/  
            L->next=s;  
        } else { flag=0; }  
    }  
}
```



单链表的基本运算—尾插法建表

- 从一个空表开始，创建一个头结点。
- 依次读取字符数组 a 中的元素，生成新结点
- 将新结点插入到当前链表的表尾上，直到结束为止。



增加一个尾指针 r ，使其始终指向当前链表的尾结点

注意：链表的结点顺序与逻辑次序相同。

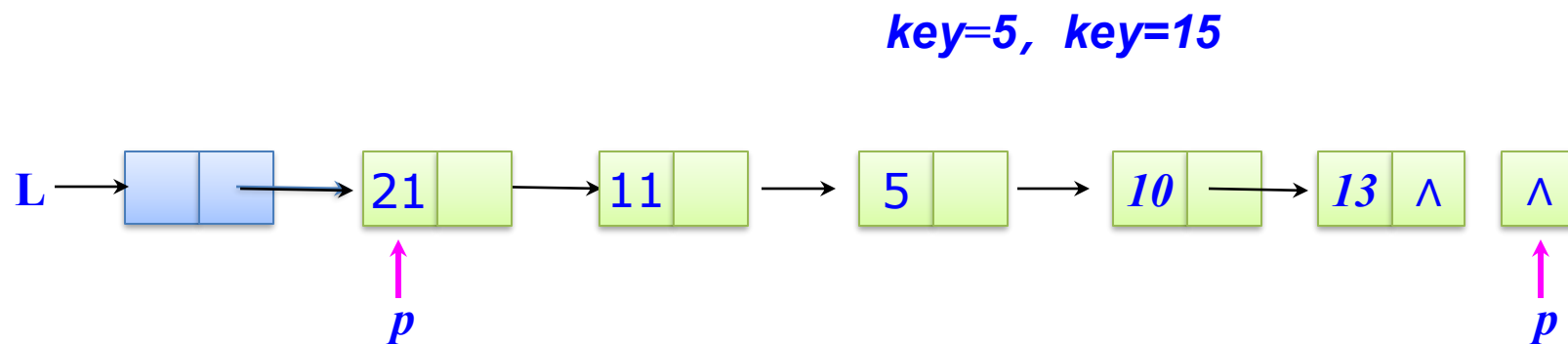


单链表的基本运算—尾插法建表

```
void CreateFromTail(LinkList L){ /* flag初值为1, 当输入“$”时, flag为0, 建表结束 */
    Node *r, *s; char c;
    int flag =1;
    r=L;          /* r 指针动态指向链表的当前表尾*/
    while(flag)
        c=getchar();
        if(c!='$') {
            s=(Node*)malloc(sizeof(Node));
            s->data=c;
            r->next=s;
            r=s;
        } else {
            flag=0;
            r->next=NULL; /*将最后一个结点的next链域置为空, 表示链表的结束*/
        }
    }
```

单链表的基本运算—单链表查找：按值

- 在带头结点的单链表L中查找其结点值等于key的结点
- 1、从链表的第一个结点出发，依次往后遍历,比较data域是否为key
- 2、如果找到则放回指针p，否则返回null。



循环条件:

1) $p \neq \text{NULL}$

返回p:

1) 找到返回p

2) 没找到返回null

`p=L->next;`

`p=p->next;`

`p->data!=key`

算法的时间复杂度为 **$O(n)$**



电子科技大学
University of Electronic Science and Technology of China

信息与软件工程学院
SCHOOL OF INFORMATION AND SOFTWARE ENGINEERING

单链表的基本运算—单链表查找：按值

/* 在带头结点的单链表L中查找其结点值等于key的结点 */

Node *Locate(LinkList L, ElemType key) { /* 若找到则返回该结点的位置p, 否则返回NULL */

Node *p;

p=L->next; /* 从表中第一个结点比较 */

while (p!=NULL)

if (p->data!=key)

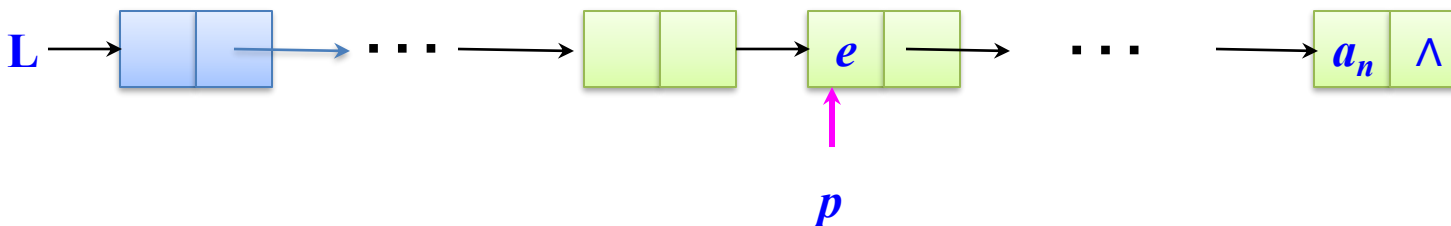
p=p->next;

else

break; /* 找到结点key, 退出循环 */

return p;

}

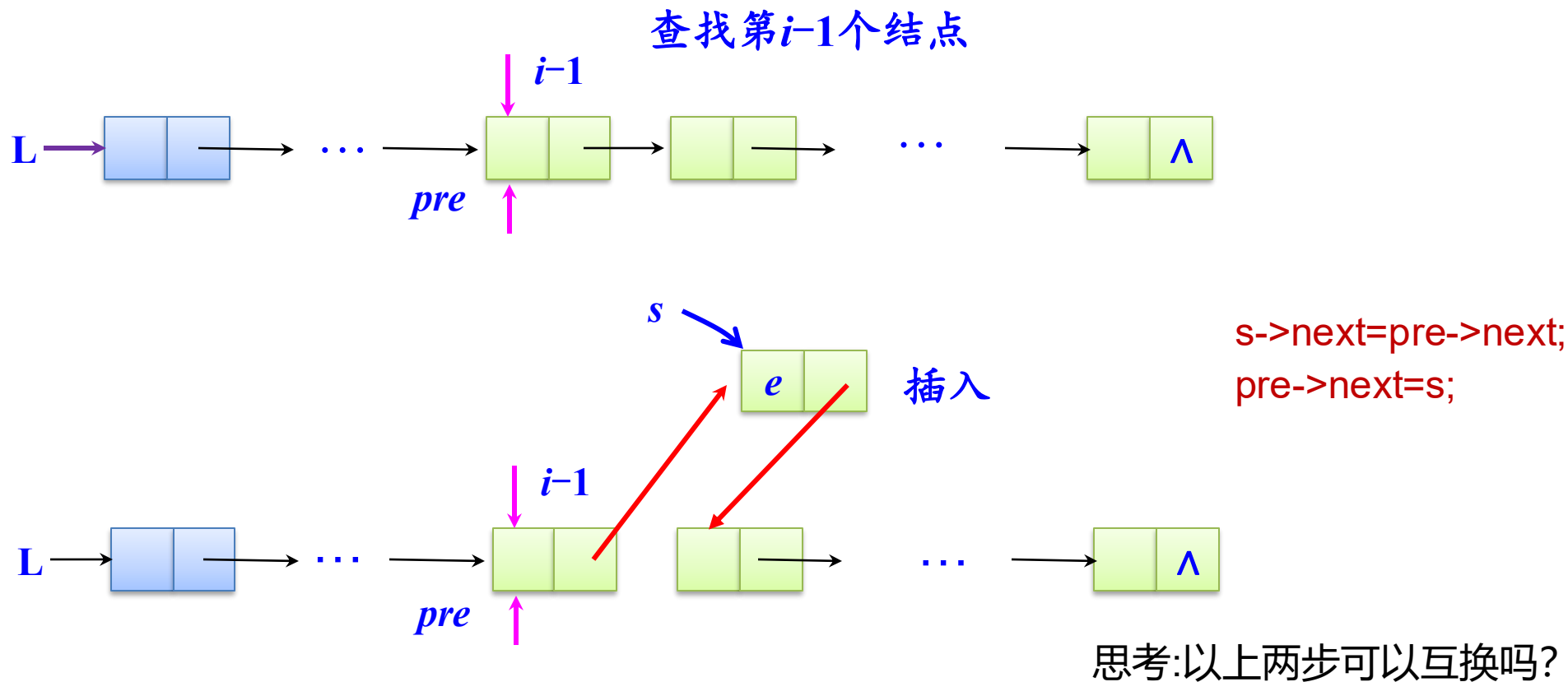


算法的时间复杂度为 **O(n)**

思考：按值查找，返回结点所在的位置？



单链表插入操作



若单链表有 m 个结点，插入位置为 $m+1$ 时，则是在尾部插入结点



单链表插入操作

```
int InsList(LinkList L,int i,ElemType e) {
```

```
    /* 在带头结点的单链表L中第i个位置插入值为e的新结点 */
```

```
    Node *pre,*s;
```

```
    int k;
```

```
    pre=L;
```

```
    k=0; /* 从头开始, 查找第i-1个结点 */
```

```
    while(pre!=NULL&& k<=(i-1)) {
```

```
        pre=pre->next;
```

```
        k++; //k=i-2时, pre指向i-1
```

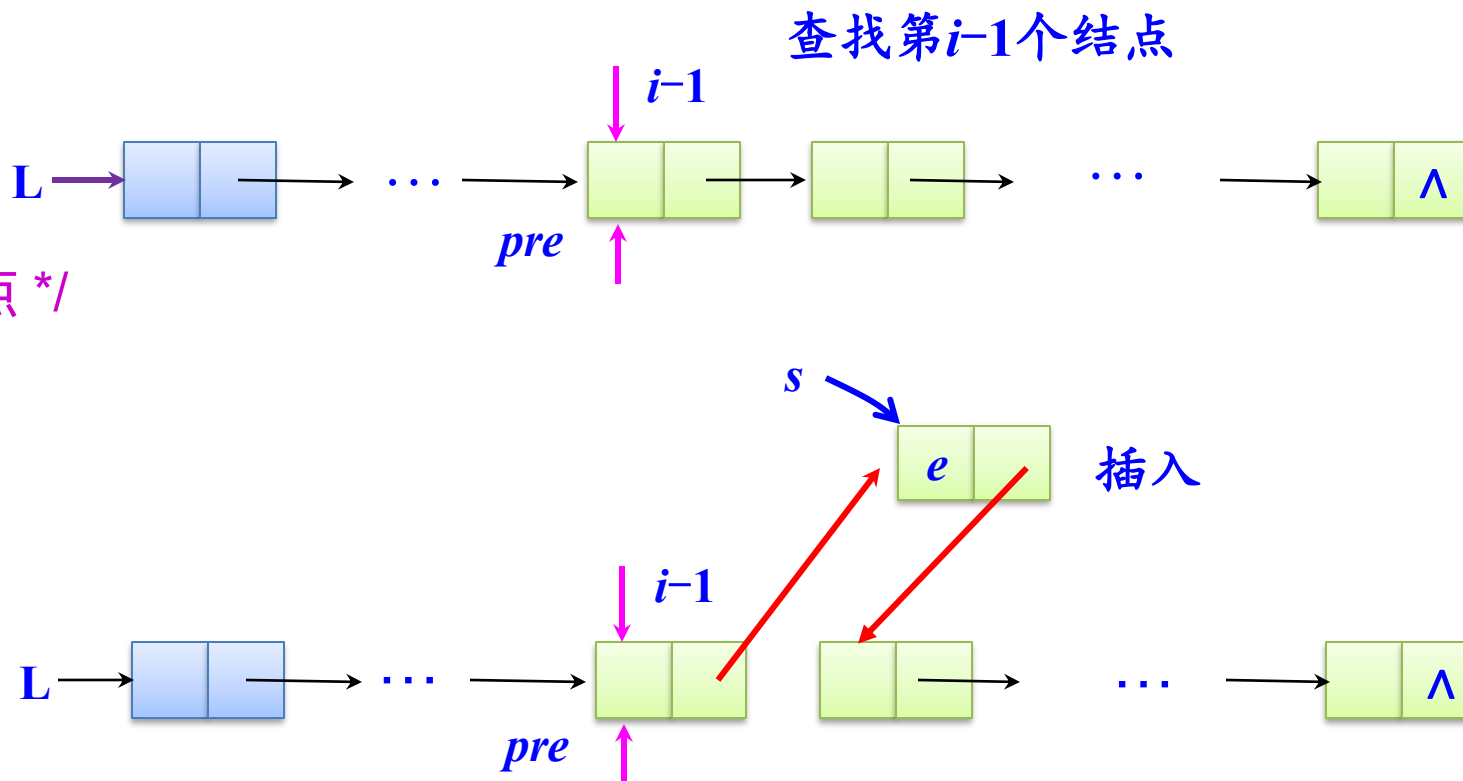
```
    } /* 查找第i-1结点 */
```

```
    if(pre==NULL)
```

```
        printf("插入位置不合理!");
```

```
        return ERROR;
```

```
    }
```



若单链表有 m 个结点, 插入位置为 $m+1$ 时, 则是在尾部插入结点



单链表插入操作

```
int InsList(LinkList L,int i,ElemType e) {
```

```
.....
```

```
s=(Node*)malloc(sizeof(Node));
```

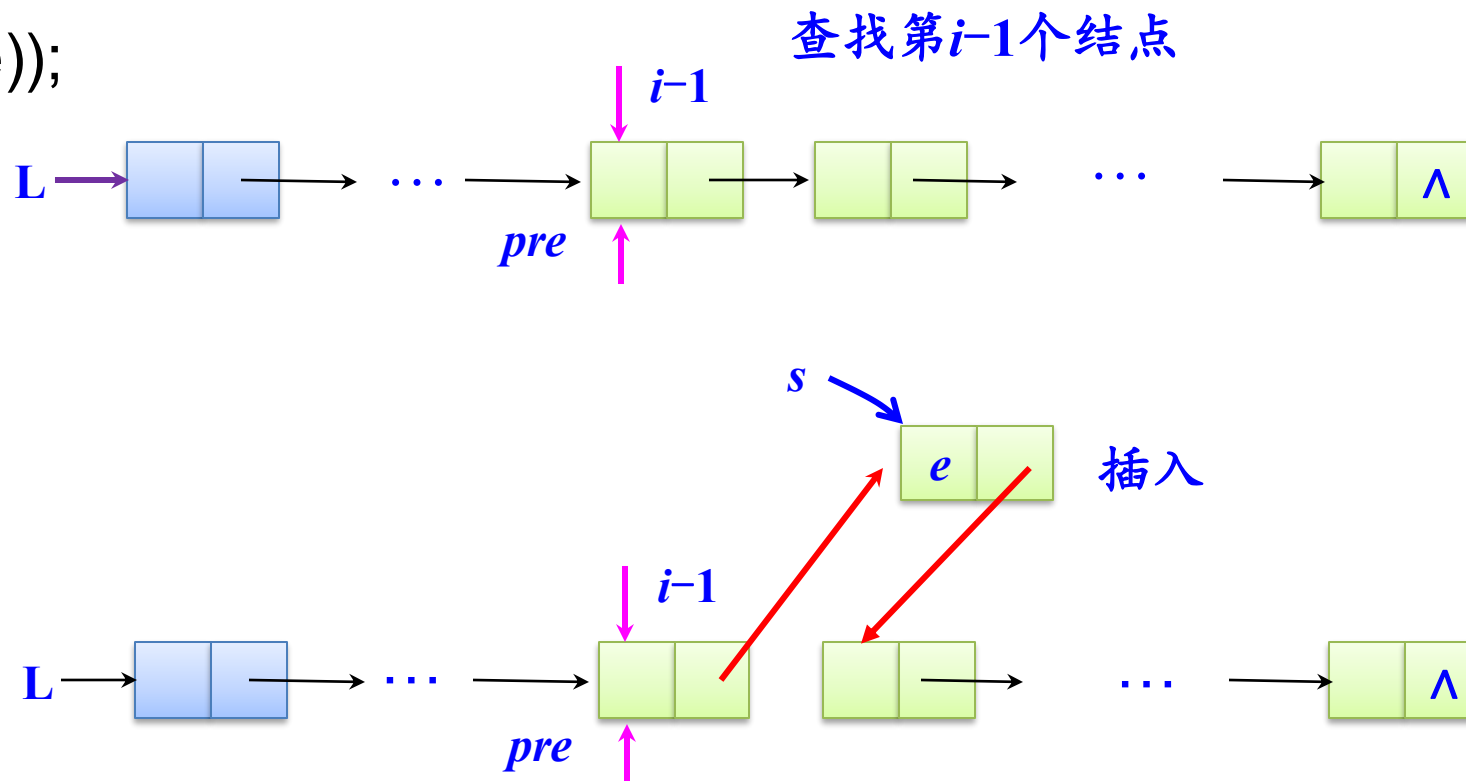
```
s->data=e;
```

```
s->next=pre->next;
```

```
pre->next=s;
```

```
return OK;
```

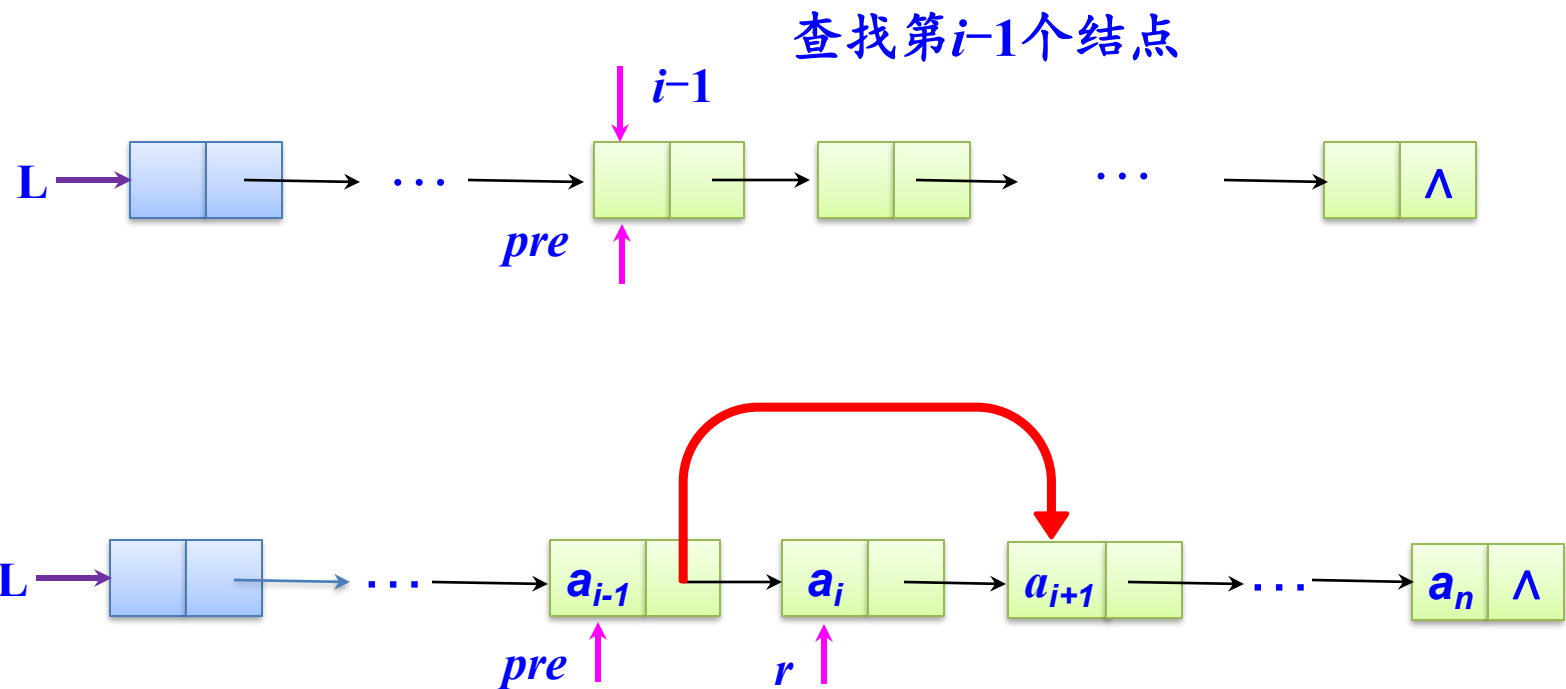
```
}
```



若单链表有 m 个结点，插入位置为 $m+1$ 时，则是在尾部插入结点



单链表删除操作



```
r = pre->next;  
pre->next = pre->next->next;  
free(r);
```



单链表删除

```
int DelList(LinkList L,int i,ElemType *e) {
```

/ 在带头结点的单链表L中删除第i个元素，并将删除的元素保存到变量*e中 */*

```
Node *pre,*r;
```

```
int k;
```

```
pre=L;
```

```
k=0;
```

```
while(pre->next!=NULL && k<i-1) {
```

```
    pre=pre->next;
```

```
    k=k+1;
```

```
} /* 查找第i-1结点 */
```

```
if((pre->next) ==NULL) {
```

```
    printf("删除结点的位置i不合理!");
```

```
    return ERROR;
```

```
}
```

```
r=pre->next;
```

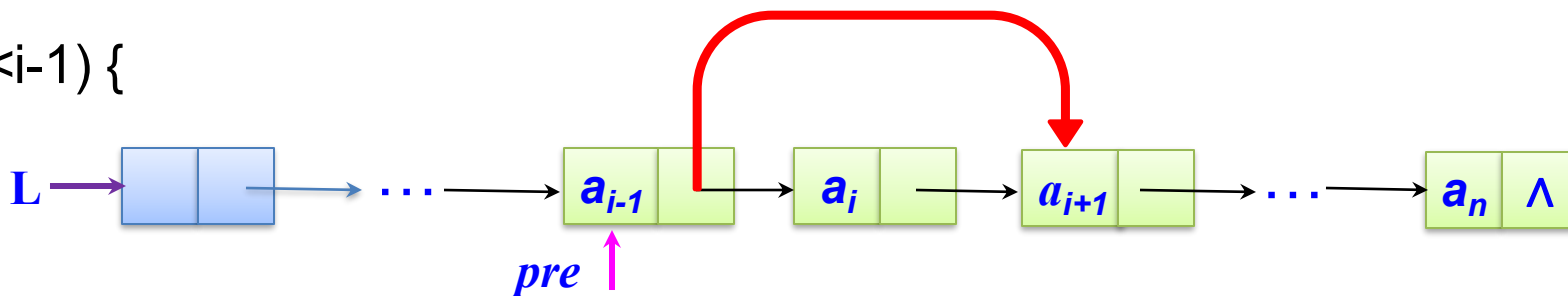
```
pre->next=pre->next->next; /*修改指针，删除结点r*/
```

```
*e = r->data;
```

```
free(r); /*释放被删除的结点所占的内存空间*/
```

```
printf("成功删除结点!");
```

```
return OK;
```



文件操作 -- 文件的打开和关闭

- 一般文件：操作前需**打开**，操作后需**关闭**。打开和关闭均通过**库函数**进行的。
- **打开文件**就是要在内存中建立缓冲区，如打开成功，打开函数返回一个内存地址值，由一个文件指针接收。以后的操作使用这个指针。若内存不可建立缓冲区，则打开失败，打开函数返回NULL。
- **关闭文件**很重要，是要将文件送回磁盘，并从内存中清除。及时释放内存空间，并可保证文件安全。
- 文件使用方式：**打开文件**-->文件读/写-->**关闭文件**



文件操作 -- 文件的打开和关闭

- `FILE *fopen(char *filename, char *mode);`
 - 参数: filename为文件名 (包括文件路径) , mode为打开方式。
 - 返回值: 成功返回指向FILE的指针, 失败返回NULL。
- `int fclose(FILE *fp);`
 - 功能: 关闭fp指向的文件, 释放文件结构占用的内存空间
 - 返回值: 正常关闭为0;出错时非0

```
FILE *fp;
```

```
if( (fp=fopen("D:\\demo.txt","rb")) == NULL ){  
    printf("Fail to open file!\\n");}
```

```
...
```

```
fclose(fp);
```



文件操作 -- 文件的打开和关闭

文件打开方式	含义
"t"	文本文件。如果不写，默认为"t"。
"b"	二进制文件。

文件读写权限	含义
"r/rb" (只读)	为 输入 打开一个文本/二进制文件
"w/wb" (只写)	为 输出 打开或建立一个文本/二进制文件
"a/ab" (追加)	向文本/二进制文件尾 追加 数据
"r+/rb+" (读写)	为读/写打开一个文本/二进制文件
"w+/wb+" (读写)	为读/写建立一个文本/二进制文件
"a+/ab+" (读写)	为读/写打开或建立一个文本/二进制文件



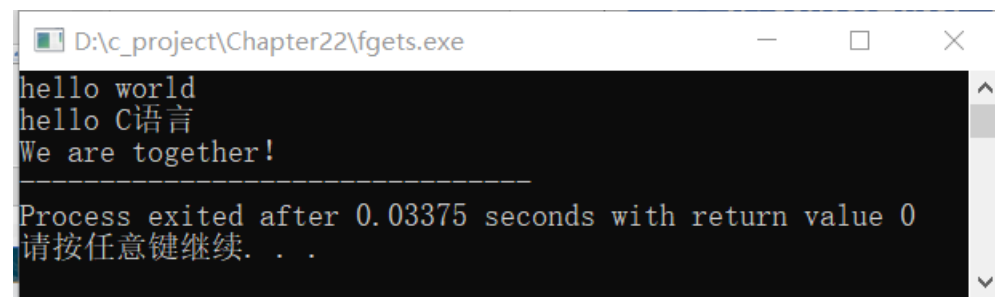
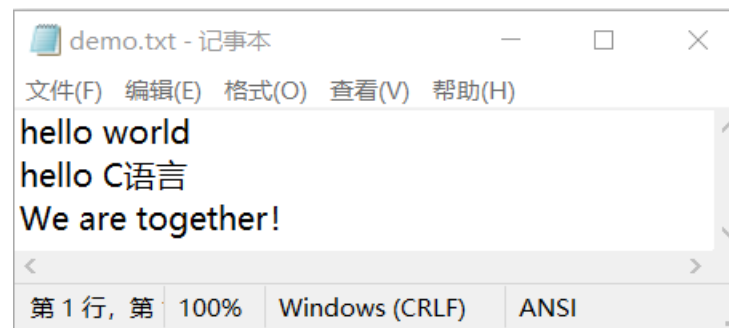
文件操作 -- 以字符串形式读写文件

- `char *fgets ( char *str, int n, FILE *fp );`
 - `str` 为字符数组, `n` 为要读取的字符数目, `fp` 为文件指针。
 - 返回值: 读取成功时返回字符数组首地址, 也即 `str`; 读取失败时返回 `NULL`;
- `int fputs( char *str, FILE *fp );`
 - `str` 为要写入的字符串, `fp` 为文件指针。
 - 写入成功返回非负数, 失败返回 `EOF`。



文件操作 -- 以字符串形式读写文件

```
#include <stdio.h>
#include <stdlib.h>
#define N 100
int main(){
    FILE *fp;
    char str[N+1];
    if( (fp=fopen("d:\\demo.txt","rt")) == NULL ){ // 打开文件
        puts("Fail to open file!");
        exit(0);}
    while(fgets(str, N, fp) != NULL){ // 以字符串形式读文件
        printf("%s", str);}
    fclose(fp); // 关闭文件
    return 0;
}
```



文件操作 -- 以数据块的形式读写文件

- `size_t fread ( void *ptr, size_t size, size_t count, FILE *fp );`
- `size_t fwrite ( void * ptr, size_t size, size_t count, FILE *fp );`
 - ptr 为内存区块的指针，它可以是数组、变量、结构体等。fread() 中的 ptr 用来存放读取到的数据，fwrite() 中的 ptr 用来存放要写入的数据。
 - size: 表示每个数据块的字节数。
 - count: 表示要读写的数据块的块数。
 - fp: 表示文件指针。
 - 理论上，每次读写 $\text{size} * \text{count}$ 个字节的数据。



文件操作 -- 格式化读写文件

- `int fscanf ( FILE *fp, char * format, ... );`
- `int fprintf ( FILE *fp, char * format, ... );`

- 举例:

```
FILE *fp;
```

```
int i, j;
```

```
char *str, ch;
```

```
fscanf(fp, "%d %s", &i, str);
```

```
fprintf(fp, "%d %c", j, ch);
```



文件操作 -- 随机读写文件

- 读写方式

- 顺序读写：位置指针按字节位置顺序移动
- 随机读写：位置指针按需要移动到任意位置

- `void rewind(FILE *fp);`

- 功能：重置文件位置指针到文件开头
- 返回值：无

- `int fseek(FILE *fp, long offset, int whence);`

- 功能：改变文件位置指针的位置
- 返回值：成功，返回0；失败，返回非0值。

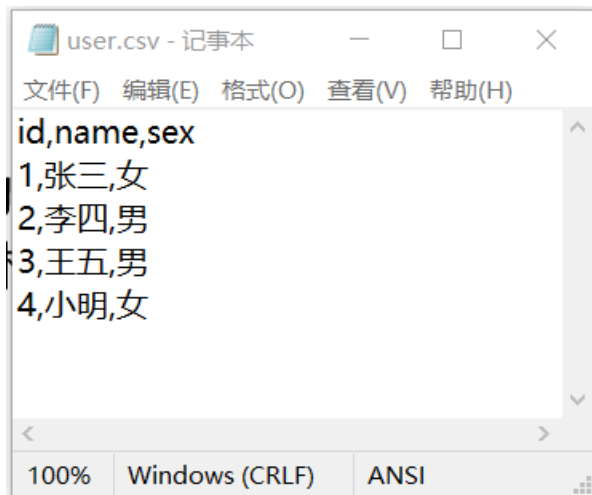
- `int ferror(FILE *fp);`

- 功能：测试文件操作是否出现错误
- 返回值：未出错，0；出错，非0



CSV 文件格式

- CSV文件可以使用excel打开，excel也可以保存成CSV格式。
- CSV 文件的第一行包含表格的列标签。随后的每一行代表表格的一行。逗号分隔行中的每个单元格。
- 下面是一个 CSV 文件的例子，该例有三列，分别标为 “id” 、name和 “sex” 。



```
user.csv - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
id,name,sex
1,张三,女
2,李四,男
3,王五,男
4,小明,女
```



训练1 结构体链表

- 从.csv文件中读取用户（id、用户名、密码、联系方式等）信息
- 以结构体链表的方式加载到内存中
- 实现用户信息的增、删、改、查
- 将修改后的用户信息写入.csv文件。



训练2 有序链表

- 假设原始文件中的数据是乱序的，思考如何创建按用户id递增的有序链表。
- 思考有序链表哪些操作（增、删、改、查）和乱序链表中的不一样，并对不一样的操作进行改写。
- 思考如何按照用户名递增的方式对链表进行排序。

